

An Empirical Study of SBOM Usage through GitHub Actions

Yo Kanemoto¹, Reika Nishimura Arakawa¹, and Mitsuaki Akiyama¹ (Member, IEEE)

¹NTT Social Informatics Labs. (e-mail: yo.kanemoto@ntt.com)

ABSTRACT Software Bill of Materials (SBOMs) are gaining attention as a key mechanism for mitigating software supply chain attacks. While many guidelines and tools have been proposed, how SBOMs are actually produced and used in practice — especially in open-source communities — remains poorly understood. In this paper, we conduct a large-scale empirical study of SBOM usage in GitHub Actions, a widely adopted CI/CD workflow platform. We first identify workflows that generate or handle SBOMs and classify their usage into four stages: generation, publication, signing, and analysis. Our analysis reveals (i) which tools are commonly used in each stage, (ii) that only 1.25% of the top-10,000 OpenSSF Criticality Score projects generate SBOMs via workflows, (iii) desirable practices such as producing SBOMs in multiple formats and programming languages, (iv) common pitfalls such as incorrect file paths that silently cause analysis of the wrong SBOM, and (v) that SBOM adoption is not monotonic—a longitudinal analysis of 2,239 workflows shows that 19.2% experience regression, oscillation, or abandonment of SBOM features over time. We also investigate projects that intentionally suppress SBOM features and clarify the reasons, such as build failures, vulnerabilities, or incompatibility with SBOM tools. These empirical findings provide practitioners and tool builders with concrete evidence about the current state and obstacles of SBOM adoption.

INDEX TERMS SBOM, software supply chain security, CI/CD, GitHub Actions, empirical study

I. INTRODUCTION

Software development today extensively reuses third-party libraries and open source software (OSS) to improve productivity. A prior study reported that roughly 90% of software projects reuse third-party components [1]. Modern software, therefore, has deep dependency chains, in which each component depends on many others. This multi-layered dependency structure is commonly referred to as the *software supply chain*.

Third-party components may contain vulnerabilities or even malicious code, which attackers can exploit. Once a compromised component enters the supply chain, the risk propagates downstream to many dependent systems. Prominent examples include the SolarWinds incident in 2020, where attackers compromised the software update process and distributed malicious updates [2], and the Log4j zero-day vulnerability disclosed in 2021, which affected a large fraction of the ecosystem [3]. These cases highlight the importance of transparency in the software supply chain.

Software Bill of Materials (SBOM) has been proposed as a foundation for such transparency [4]. An SBOM is a structured inventory of components, libraries, and dependencies that constitute a software product. With SBOMs, software producers and consumers can more easily conduct security

and license assessments, such as checking whether vulnerable components are included.

Although the importance of SBOMs is widely discussed in standards, whitepapers, and community efforts, how SBOMs are actually adopted and used in OSS development remains unclear. Existing work primarily relies on interviews or discussions in issue trackers, and large-scale quantitative analyses of *implemented* SBOM workflows are still scarce.

In this paper, we focus on GitHub Actions, GitHub's CI/CD platform, as a practical setting for generating and consuming SBOMs. By analyzing workflow files of OSS projects, we aim to answer the following research questions (RQs):

- **RQ1:** What tools are commonly used when working with SBOMs, and in which stages of the workflow are they used?
- **RQ2:** To what extent are SBOMs adopted in security-critical OSS projects, and what characteristics differentiate projects that generate SBOMs?
- **RQ3:** What common mistakes and desirable practices can be observed when SBOMs are generated and used in workflows?
- **RQ4:** Why do projects suppress SBOM features in their workflows?
- **RQ5:** How do SBOM workflows evolve?

For RQ1, we collect SBOM-related workflows through keyword search and LLM-based classification, and categorize them into four usage stages: generation, publication, signing, and analysis. For RQ2, we investigate the top 10,000 projects ranked by the OpenSSF Criticality Score to examine SBOM adoption in security-critical OSS projects. For RQ3, we use LLM-assisted analysis combined with manual inspection to identify common mistakes and desirable practices in SBOM workflows. For RQ4, we analyze workflow comments and related artifacts to investigate why projects suppress SBOM features such as generation, publication, or signing. For RQ5, we conduct a longitudinal analysis of monthly workflow snapshots to track how SBOM features are adopted, maintained, or abandoned over time.

To answer these questions, we first identify workflows potentially related to SBOMs using a keyword-based search. We then employ a large language model (LLM) to classify each workflow into SBOM usage stages (generation, publication, signing, analysis) [5]. Finally, we conduct a detailed manual and automated analysis of the resulting dataset, as well as of the top 10,000 projects according to the OpenSSF Criticality Score [6].

Our contributions are as follows:

- We provide, to the best of our knowledge, the first large-scale empirical study of SBOM usage in GitHub Actions workflows, quantifying which tools and patterns are prevalent at each stage of use.
- We show that only a small fraction (1.25%) of critical OSS projects generate SBOMs via CI/CD workflows, and SBOM-related practices such as publication, signing, and analysis are even less common.
- We identify recurring mistakes (e.g., wrong file paths) and good practices (e.g., multi-format and multi-language SBOM generation), as well as practical obstacles that hinder SBOM adoption.
- We conduct a longitudinal analysis of 2,239 workflows and show that SBOM adoption is not monotonic: 19.2% of workflows experience regression, oscillation, or abandonment over time.

II. BACKGROUND

A. SOFTWARE BILL OF MATERIALS

An SBOM is a machine-readable document that lists the components, libraries, and dependencies that make up a software product. It typically includes information such as component names, versions, suppliers, and the dependency relationships between components. By providing such an inventory, SBOMs support tasks such as vulnerability management, license compliance, and impact analysis when new vulnerabilities are disclosed.

Two major SBOM formats are widely used in practice today: *CycloneDX* and *Software Package Data Exchange* (SPDX). CycloneDX is an OWASP project that defines an SBOM specification in formats such as XML and JSON [7]. SPDX is a Linux Foundation project that provides a machine-

readable format, originally focused on license information but now extended to general metadata about packages [8].

In addition to the SBOM content itself, some ecosystems encourage attaching digital signatures or attestations to SBOM files. Signing allows consumers to verify the integrity and provenance of SBOMs, ensuring that they are not tampered with and are produced by the expected party.

B. GITHUB ACTIONS

GitHub Actions is a CI/CD platform tightly integrated into GitHub repositories [9]. Workflows are specified in YAML files under the `.github/workflows` directory and are triggered by events such as `push` or `pull_request`. Figure 1 shows a sample workflow configuration. A workflow consists of one or more jobs, and each job consists of a series of steps that either invoke predefined *actions* or execute arbitrary shell commands. Actions are reusable, packaged tasks (e.g., checking out code) that are published on the GitHub Marketplace and can be easily incorporated into workflows.

```

1 name: ...           # workflow name
2 on: ...             # trigger event
3 jobs:
4   build:             # job name
5     runs-on: ubuntu-latest # runner
6     steps:
7       - name: Check out code
8         uses: actions/checkout@v4 # action
9       - run: echo "Hello, world" # command

```

FIGURE 1: Example workflow configuration.

Because workflows are executed in response to code changes or release events, they are a natural place to automate SBOM generation and consumption. By generating SBOMs as part of CI/CD, projects can keep SBOMs synchronized with the current build, addressing the challenge of keeping SBOMs up to date [10]. Moreover, publishing or signing SBOMs during the pipeline can make them easily consumable by downstream users and security tools.

GitHub Actions has rapidly become the dominant CI/CD platform for OSS, covering over 50% of CI-using repositories [11], [12], and major SBOM tools provide first-class GitHub Actions integrations. We discuss the limitation of platform-specific scope in Section X.

III. METHODOLOGY: MINING SBOM USAGE IN GITHUB ACTIONS

This section describes how we constructed our dataset and how we classified SBOM-related usage. Figure 2 illustrates the overall methodology of our study. We first use keyword-based search to collect potentially SBOM-related workflows, then classify each workflow into SBOM usage stages using an LLM, and finally perform detailed analysis on the resulting dataset.

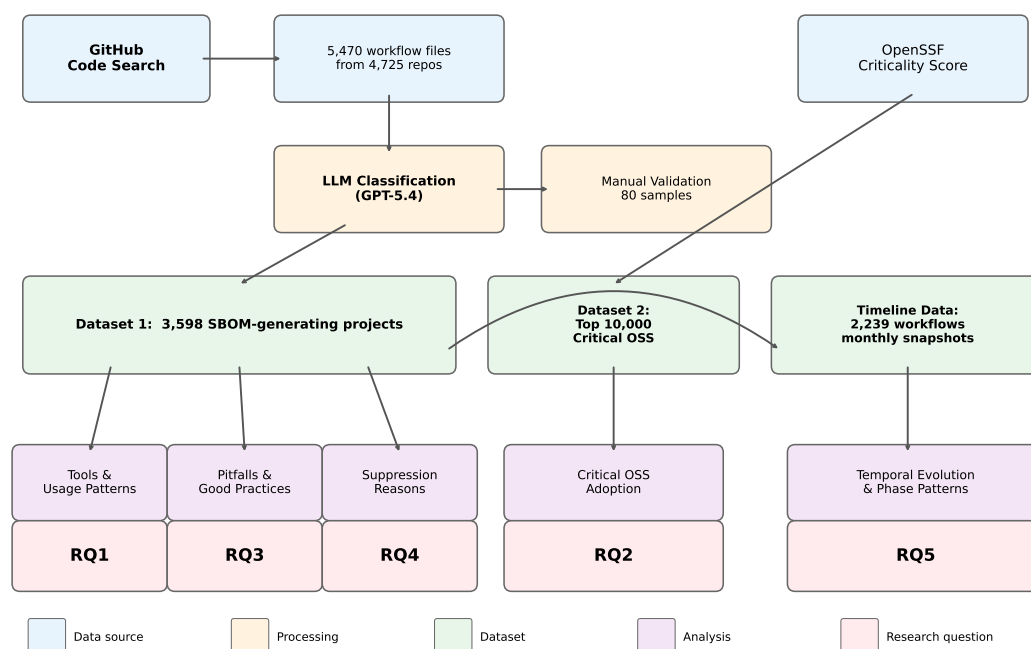


FIGURE 2: Overview of the study methodology. GitHub code search yields candidate workflows, which are classified by an LLM into four SBOM usage stages (validated by manual inspection of 80 samples). Dataset 1 (SBOM-generating projects) feeds RQ1 (tools and patterns), RQ3 (pitfalls and practices), and RQ4 (suppression reasons). Dataset 2 (critical OSS) feeds RQ2. Longitudinal timeline analysis of monthly workflow snapshots feeds RQ5 (temporal evolution).

A. DATASET CONSTRUCTION

We first used GitHub code search to identify workflow files that potentially work with SBOMs. Our search targeted files under the `.github/workflows` directory that satisfy the following conditions: (i) the file is written in YAML; (ii) the repository is not a fork and not archived; and (iii) the file contains SBOM-related keywords such as “SBOM” or “SPDX”. This yielded 5,470 unique workflow files from 4,725 OSS projects.

For more precise identification of SBOM-related workflows, we used an LLM (OpenAI GPT-5.4) to determine whether each workflow actually generates SBOMs. We provided the workflow content and an instruction prompt that asked whether the workflow performs SBOM generation, signing, publication, or analysis, and requested JSON output with Boolean flags for each category. The prompt template is shown below:

Example prompt judge SBOM-related workflows

```

You are an expert software developer.
## Task
Analyze the given GitHub Actions
workflow from the following
perspectives:
- Does it generate an SBOM?
- Does it sign an SBOM to make it
verifiable?
- Does it publish the generated SBOM?
- Does it analyze the generated SBOM?
## Constraints
- Output the result in JSON format.
- Include the relevant workflow
content verbatim as evidence.
## GitHub Actions Workflow
{workflow content}

```

As a sanity check, we manually inspected 20 randomly sampled workflows for each of the four classification stages (generation, publication, signing, and analysis) by examining the workflow YAML and the LLM’s classification reason. For each sample, we extracted the lines matching stage-specific keywords (e.g., `sbom-action` for generation, `upload-artifact` for publication, `cosign attest` for signing, `grype sbom:` for analysis) and compared them with the LLM’s judgment. For generation, we sampled

20 positive classifications to verify that the workflows indeed generate SBOMs; for the other three stages, we sampled 10 positive and 10 negative classifications each. All 80 inspected classifications were confirmed to be correct, giving us confidence in the LLM's classification quality.

Based on the classification, we identified 3,598 projects whose workflows include at least one step that generates an SBOM. We refer to this subset as *Dataset 1*. Table 1 summarizes the dataset.

TABLE 1: Overview of Dataset 1.

Item	Value
Workflow files (keyword search)	5,470
OSS projects (keyword search)	4,725
OSS projects (SBOM generation)	3,598
Collection period	Jan–Mar 2026

B. SBOM USAGE STAGES

Following practice and prior work, we classify SBOM usage into four stages:

a: SBOM generation

A workflow is labeled as *generation* if it includes a step that creates an SBOM file, either by invoking a dedicated SBOM tool or by enabling SBOM output in existing tools. For example, the following step uses Anchore's `sbom-action` to generate an SPDX JSON SBOM:

```
1 - name: Generate SBOM
2   uses: anchore/sbom-action
3   with:
4     format: spdx-json
5     output-file: sbom.spdx.json
```

b: SBOM publication

We label workflows as *publication* when they expose SBOM files to external consumers, e.g., by uploading them as CI artifacts or by attaching them to GitHub releases. The following example uploads `sbom.spdx.json` as an artifact:

```
1 - name: Upload SBOM artifact
2   uses: actions/upload-artifact@v4
3   with:
4     name: sbom
5     path: sbom.spdx.json
```

c: SBOM signing

Workflows in this category attach signatures or attestations to SBOMs, enabling consumers to verify integrity and provenance. For instance, the following step uses GitHub's `attest-sbom` action:

```
1 - name: Attest SBOM
2   uses: actions/attest-sbom@v1
3   with:
4     sbom-path: sbom.spdx.json
```

d: SBOM analysis

Finally, we label workflows as *analysis* if they feed SBOMs into vulnerability or license scanners, or verify SBOM signatures. An example is uploading an SBOM to a DependencyTrack server for vulnerability analysis:

```
1 - uses: actions/download-artifact@v4
2   with:
3     name: sbom.spdx.json
4 - name: Upload SBOM to DependencyTrack
5   run: curl -X POST
6         https://.../api/v1/bom \
          -F "bom=@sbom.spdx.json"
```

Using these definitions, we analyze which tools are used and how these stages overlap.

IV. RESULTS: TOOLS AND USAGE PATTERNS (RQ1)

All 3,598 projects in Dataset 1 generate SBOMs by definition. Among these, 2,359 (65.6%) publish SBOMs, 673 (18.7%) sign SBOMs, and 135 (3.8%) use SBOMs for downstream analysis. Note that a single project may use multiple tools, so the per-tool counts in the following tables can overlap.

A. SBOM GENERATION TOOLS

Table 2 summarizes the most frequently used actions and commands for SBOM generation. The dominant tools are Syft and Trivy, both widely used software composition analysis (SCA) tools, and Docker build options that emit SBOMs alongside container images. We also observe language-specific tools, such as CycloneDX generators for Go, Python, Java, and other ecosystems.

TABLE 2: Top actions and commands for SBOM generation (Dataset 1, $n = 3,598$ projects). Counts may overlap because a project can use multiple tools. Docker `buildx` with `-sbom=true` also generates SBOMs but is omitted here because it is classified as a build option rather than a dedicated SBOM tool.

Action	#Projects (%)
anchore/sbom-action	1,101 (30.6%)
aquasecurity/trivy-action	298 (8.3%)
CycloneDX/gh-gomod-generate-sbom	44 (1.2%)
Command	#Projects (%)
syft ...	448 (12.5%)
cyclonedx-py / cyclonedx-bom ...	241 (6.7%)
cdxgen ...	214 (5.9%)
cargo cyclonedx ...	114 (3.2%)
sbom-tool (Microsoft) ...	86 (2.4%)

These results suggest that SBOM generation is typically implemented either by general-purpose SCA tools or by language-specific CycloneDX generators.

B. SBOM PUBLICATION

Among 3,598 SBOM-generating projects, 2,359 (65.6%) publish SBOMs explicitly. Table 3 lists the top actions used for SBOM publication. The most common pattern is to upload SBOMs as build artifacts via

actions/upload-artifact. Another common pattern is to attach SBOMs to GitHub releases using actions such as softprops/action-gh-release. We note that attestation-based registry pushes (actions/attest-sbom, cosign attest) are classified under *signing* rather than publication, as they primarily serve integrity verification rather than distribution.

TABLE 3: Top actions for SBOM publication ($n = 2,359$ publishing projects). Counts may overlap because a project can use multiple actions.

Action	#Projects (%)
actions/upload-artifact	1,500 (63.6%)
softprops/action-gh-release	122 (5.2%)
actions/upload-release-asset	39 (1.7%)
DependencyTrack/gh-upload-sbom	20 (0.8%)
svenstaro/upload-release-action	10 (0.4%)

Among projects that publish SBOMs, the majority use build artifacts rather than release assets, making it harder for consumers to associate SBOMs with specific software versions. Attaching SBOMs to releases is a recommended practice, but only a minority of SBOM-publishing projects have adopted it. However, the remaining 34.4% of SBOM-generating projects do not expose SBOMs to external consumers at all, which limits their usefulness. For SBOMs to serve their intended purpose of enhancing software transparency, they must be accessible to downstream users. We encourage projects to publish SBOMs alongside their software releases as a standard practice.

C. SBOM SIGNING

Among 3,598 SBOM-generating projects, 673 (18.7%) sign or attest their SBOMs. Table 4 shows the top tools used for SBOM signing. We classify only SBOM-specific attestations (cosign attest, actions/attest-sbom, syft attest) as signing; general build provenance attestation (attest-build-provenance) and Docker image signing (cosign sign) are excluded. GitHub's built-in actions/attest-sbom is the most widely used (54.4%), followed by Sigstore's Cosign (48.6% via the installer action, 27.3% via the CLI).

TABLE 4: Top actions and commands for SBOM signing ($n = 673$ signing projects). Counts may overlap because a project can use multiple tools.

Action	#Projects (%)
actions/attest-sbom	366 (54.4%)
sigstore/cosign-installer	327 (48.6%)
actions/attest	33 (4.9%)
Command	#Projects (%)
cosign attest ...	184 (27.3%)
syft attest ...	2 (0.3%)

Sigstore's focus on reducing key management overhead appears to have contributed to its popularity in OSS communities.

D. SBOM ANALYSIS

Among 3,598 SBOM-generating projects, 135 (3.8%) download and use SBOMs for downstream analysis. We classify a project as performing SBOM analysis only when the SBOM file is explicitly used as input (e.g., grype sbom:, trivy sbom); running vulnerability scans directly on source code or container images does not count. We identified three main purposes for SBOM analysis: vulnerability scanning, license checking, and signature verification (Table 5). For vulnerability scanning, we observed tools such as Grype and Trivy used with SBOM files as input, as well as DependencyTrack for centralized SBOM-based vulnerability management.

TABLE 5: Representative SBOM analysis use cases and tools.

Purpose	Type	Example tools
Vuln. scan	Cmd	grype sbom:sbom.json
	Cmd	trivy sbom ...
	Act	DependencyTrack/gh-upload-sbom
License check	Cmd	trivy sbom -scanners license
	Cmd	python3 license-check.py -sbom
	Act	mvdclajn/licenses-action
Sig. verify	Cmd	cosign verify-attestation

E. OVERLAP OF USAGE STAGES

Figure 3 illustrates the overlap between SBOM generation and publication. As noted in Section IV, the majority of generators (2,359; 65.6%) also publish SBOMs, but a substantial minority does not.

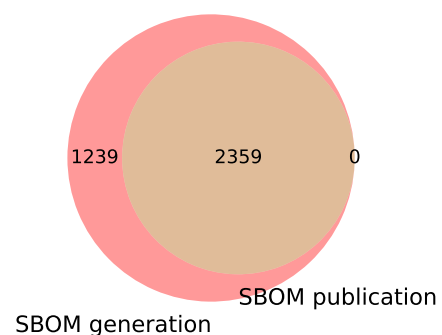


FIGURE 3: Overlap between SBOM generation and publication in Dataset 1.

Figure 4 shows the overlap among publication, signing, and analysis. When considering all three stages together, only 19 projects (0.5% of generators) implement publication, signing, and analysis in addition to generation. Thus, multi-stage SBOM usage is still rare.

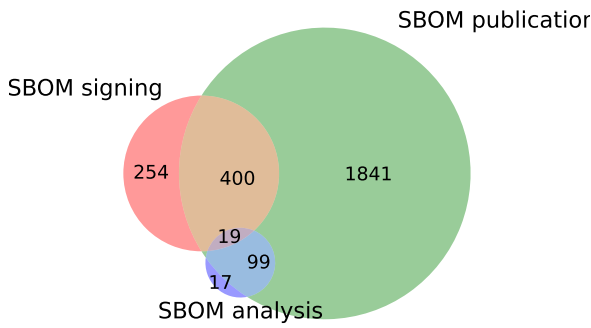


FIGURE 4: Overlap among SBOM publication, signing, and analysis in Dataset 1.

Answer to RQ1

RQ1: What tools are commonly used when working with SBOMs, and in which stages of the workflow are they used? For SBOM generation, SCA tools such as Syft (30.6%), Trivy (8.3%), and cdxgen (5.9%) are widely used. For publication, `actions/upload-artifact` dominates (63.6%); for signing, GitHub's `actions/attest-sbom` (54.4%) and Sigstore's Cosign (48.6%) are the two leading tools. In contrast, no single tool dominates SBOM analysis, where the choice depends on the specific purpose (vulnerability scanning, license checking, or signature verification). Among 3,598 SBOM-generating projects, 65.6% (2,359) also publish SBOMs, and only 0.5% (19) practice all of generation, publication, signing, and analysis.

V. SBOM USAGE IN CRITICAL OSS PROJECTS (RQ2)

To understand SBOM adoption among particularly important OSS projects, we analyzed the top-10,000 projects ranked by the OpenSSF Criticality Score [6]. This score aggregates metrics such as activity, number of contributors, and dependency relationships into a value between 0 and 1. Higher scores indicate projects on which many others depend and whose compromise would have large impact. The score range for the top-10,000 projects was [0.56, 0.93], with star counts ranging from 19 to 408,068 and contributor counts ranging from 1 to 5,000. We applied the same LLM-based classification as in the previous section to determine whether each project's workflows generate SBOMs. We refer to this set of 10,000 projects as *Dataset 2*.

Among these 10,000 projects, only 125 (1.25%) have workflows that generate SBOMs. For comparison, Novikov and Riehle [13] found policy-driven SBOM *files* in only 0.56% of 26,823 repositories, while Kubo et al. [14] reported an average implementation rate of 17.5% across five GitHub-recommended security practices. SBOM adoption thus lags well behind other security measures, though it exceeds the rate of static SBOM file provision.

We further compared basic project characteristics between

SBOM-generating and non-generating projects (Figure 5). Because repository metrics typically follow heavy-tailed distributions, we use the Mann–Whitney U test for statistical significance and Cliff's delta (δ) for effect size, following established guidelines in software engineering research ($|\delta| < 0.147$: negligible, < 0.33 : small, < 0.474 : medium, ≥ 0.474 : large).

All three metrics show statistically significant differences ($p < 0.05$) between SBOM-generating and non-generating projects. However, the effect sizes vary considerably. Commit frequency exhibits a medium effect ($\delta = 0.45$), indicating that SBOM adoption is most strongly associated with development activity rather than popularity or team size. Star count shows a small effect ($\delta = 0.21$), while contributor count is negligible ($\delta = 0.14$), meaning the two groups are practically indistinguishable in terms of team size. These results suggest that SBOM adoption is primarily driven by active development practices rather than project scale, possibly because frequently releasing projects have stronger incentives to automate supply chain transparency. Even among critical projects, SBOM usage is still limited and far from ubiquitous.

Answer to RQ2

RQ2: To what extent are SBOMs adopted in security-critical OSS projects, and what characteristics differentiate projects that generate SBOMs? Among the top-10,000 OpenSSF Criticality Score projects, only 1.25% generate SBOMs via workflows. Mann–Whitney U tests show that commit frequency is the strongest differentiator (Cliff's $\delta = 0.45$, medium effect), while contributor count is practically indistinguishable ($\delta = 0.14$, negligible). SBOM adoption appears driven by active development practices rather than project scale.

VI. PITFALLS AND GOOD PRACTICES (RQ3)

A. COMMON MISTAKES

We distinguish *configuration errors*—concrete misconfigurations that cause SBOM workflows to malfunction—from *best-practice gaps* such as not signing or not publishing SBOMs, which represent incomplete adoption rather than errors. The LLM flagged issues in 2,870 out of 3,598 projects (79.8%), but the majority were best-practice gaps. After filtering for actual configuration errors, we identified 1,417 projects (39.4%) with at least one concrete misconfiguration. Table 6 summarizes the most common categories. These projects are still counted in the generation, publication, signing, and analysis statistics above; having a mistake does not exclude a project from those categories.

TABLE 6: Common SBOM configuration errors ($n = 3,598$ projects).

Error category	#Projects (%)
Path mismatch between steps	828 (23.0%)
<code>continue-on-error</code> masking failures	243 (6.8%)
Duplicate/redundant SBOM generation	168 (4.7%)

The most prevalent error is path mismatch (23.0%), where the SBOM file path used in a downstream step does not match

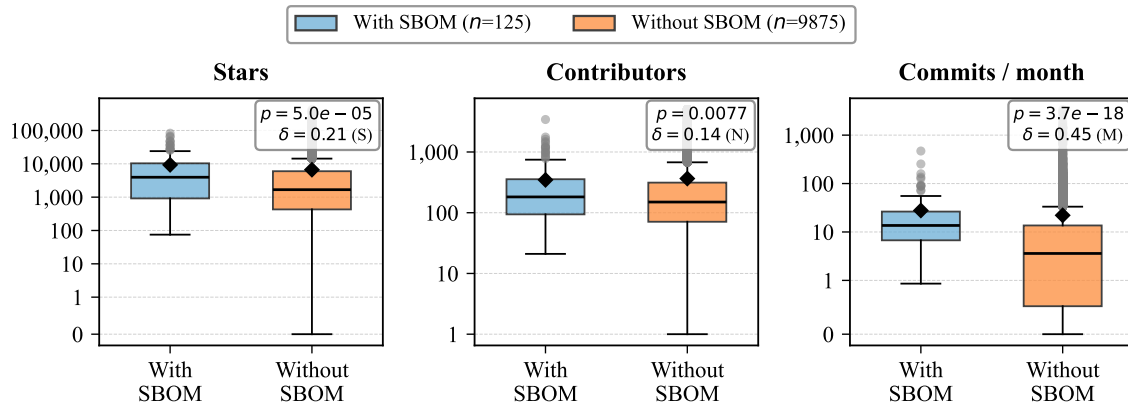


FIGURE 5: Comparison of critical projects with and without SBOM generation ($n_{\text{SBOM}} = 125$, $n_{\text{non}} = 9,875$). The y-axis uses a symmetric log scale. Each panel shows the p -value (Mann–Whitney U) and Cliff’s δ (S = small, N = negligible, M = medium). Diamond markers indicate means.

the path produced by the generation step. A typical example is inconsistent file paths between generation and analysis steps.

The snippet below illustrates such a case. The workflow generates an SBOM with `sbom4python` and uploads it as an artifact, but later the analysis step refers to a different path:

```
1 - name: SPDX: Generate SBOM
2   run: sbom4python -o
      /tmp/sbom4python.spdx.json ...
3 - name: Upload SBOM
4   uses: actions/upload-artifact@v4
5   with:
6     name: sbom4python
7     path: "/tmp/sbom4python.*.json"
8 - name: Generate Summary
9   run: SBOM4PYTHON_SBOM_PATH=\
10      "python-trivy/trivy.spdx.json" \
11      sbomqs score
      $SBOM4PYTHON_SBOM_PATH -j
```

Here, the analysis uses a different path (`python-trivy/trivy.spdx.json`) instead of the SBOM generated by `sbom4python`. Such mistakes may silently lead to analyzing the wrong SBOM or none at all.

B. DESIRABLE PRACTICES

We also observed several desirable practices that enhance SBOM usefulness (Table 7). The most common is to generate SBOMs in both SPDX and CycloneDX formats (315 projects, 8.8%). Given the coexistence of multiple formats, offering both improves compatibility with diverse downstream tools.

The second most common is to automatically update SBOMs in the repository when build-time SBOMs change (29 projects, 0.8%), for example by committing SBOM diffs back to the repository. This helps users track fine-grained changes in dependencies over time.

A smaller number of projects generate SBOMs for multiple programming languages used in a project (13 projects, 0.4%),

ensuring that all components are covered regardless of the language ecosystem.

Finally, 11 projects (0.3%) validate or score the quality of generated SBOMs using tools such as `cyclonedx-cli validate` (schema validation) or `sbomqs score` (quality scoring against criteria such as the NTIA minimum elements).

TABLE 7: Examples of good SBOM practices ($n = 3,598$).

Practice	#Projects (%)
Generate both SPDX and CycloneDX	315 (8.8%)
Update stored SBOMs based on diffs	29 (0.8%)
Support multiple languages	13 (0.4%)
Validate or score SBOM quality	11 (0.3%)

Answer to RQ3

RQ3: What common mistakes and desirable practices can be observed when SBOMs are generated and used in workflows? We identified 1,417 projects (39.4%) with concrete configuration errors, the most common being path mismatches between SBOM generation and consumption steps (23.0%), continue-on-error masking SBOM failures (6.8%), and duplicate/redundant SBOM generation (4.7%). Desirable practices include generating SBOMs in both SPDX and CycloneDX formats for broader compatibility and generating SBOMs for multiple programming languages to improve coverage.

VII. REASONS FOR SUPPRESSING SBOM FEATURES (RQ4)

We examined projects that explicitly *suppress* one or more SBOM-related features in their workflows. We define *SBOM feature suppression* as any mechanism that intentionally disables or bypasses an SBOM usage stage (generation, publication, signing, or analysis) that would otherwise be active. This includes both full suppression (e.g., removing SBOM generation entirely) and partial suppression (e.g., generating SBOMs but disabling their publication). The LLM identified 996 projects (27.7%) with at least one suppression mechanism. Through qualitative analysis of workflow comments,

issue discussions, and commit messages, we classified the barriers into four categories (Table 9).

A. SUPPRESSION MECHANISMS

We first examined *how* projects suppress SBOM features. Table 8 shows the most common mechanisms, categorized by which SBOM stage they affect. The most prevalent mechanism is suppressing SBOM publication via `upload-artifact: false` (113 occurrences), meaning SBOMs are generated but never shared (publication suppression). The second most common is commenting out SBOM-related steps (94 occurrences), suggesting incomplete or work-in-progress integration (generation suppression). Signing suppression via `provenance: false` (61 occurrences) disables build provenance attestation in Docker builds.

TABLE 8: Common mechanisms for suppressing SBOM features, categorized by affected stage ($n = 996$ projects with suppression). These six mechanisms cover 268 unique projects (26.9%); counts may overlap because a project can use multiple mechanisms.

Stage	Mechanism	Count
Publication	<code>upload-artifact: false</code>	113
Generation	Commented-out steps	94
Signing	<code>provenance: false</code>	61
Generation	<code>if: false</code> (entire job)	7
Generation	<code>sbom: false</code>	5
Generation	<code>SKIP_SYFT / SKIP_CDXGEN</code>	2

B. REASONS FOR SUPPRESSION

To understand *why* projects suppress SBOM features, we examined only cases where the workflow contains an explicit comment, TODO marker, or documented reason for the suppression. Of the 996 projects with suppression mechanisms, 363 (36.4%) contained such explicit documentation in their workflow files, which we classified into four categories (Table 9). The remaining 633 projects had suppression mechanisms (e.g., `if: false`, conditional flags) but no documented reason.

TABLE 9: Categorized reasons for suppressing SBOM features ($n = 363$ unique projects out of 996 with suppression). Only projects whose workflow comments or documentation explicitly state the reason are included. Per-category counts sum to 430 because 64 projects belong to two or more categories.

ID	Reason	#Projects
R1	Conditional / cost-driven suppression (PR skip, optional, resource saving)	256
R2	Incomplete setup / work-in-progress (commented-out steps, TODO)	105
R3	Platform incompatibility (GHES, <code>provenance: false</code>)	54
R4	Tool vulnerabilities, bugs, and build errors (e.g., cyclonedx-node-npm issue #1224, syft issue #1622)	15

a: R1: Conditional and cost-driven suppression (256 projects).

This is the most prevalent category. A common pattern is skipping SBOM steps on `pull_request` events to save resources, generating SBOMs only for release builds. Other projects expose SBOM generation as a workflow input that defaults to `false`, requiring explicit opt-in. Cost and resource concerns also drive suppression: some projects explicitly cite CI minutes or storage costs as reasons for making SBOM steps optional.

b: R2: Incomplete setup (105 projects).

These projects contain commented-out steps, TODO comments, or explicit mentions that configuration is not yet ready. For example, several projects comment out `DependencyTrack` uploads or note that signing keys are pending setup, indicating awareness of the need but incomplete implementation.

c: R3: Platform incompatibility (54 projects).

GitHub Enterprise Server (GHES) does not support certain actions such as `actions/attest-sbom`, forcing enterprise users to disable attestation. Many projects set `provenance: false` in Docker builds due to compatibility issues with multi-platform builds or registry constraints.

d: R4: Tool vulnerabilities, bugs, and build errors (15 projects).

These projects disable SBOM features due to specific tool issues documented in workflow comments, often referencing upstream GitHub issues. Examples include disabling an SBOM job because the `CycloneDX` npm package has a reported vulnerability (`cyclonedx-node-npm` #1224), skipping Syft due to a known error (`syft` #1622), and working around a `buildx` bug (`build-push-action` #461).

C. CONNECTION TO BROADER SECURITY PRACTICE BARRIERS

These findings are suggestive of parallels with the barriers identified by Kubo et al. [14] for general GitHub Actions security practices: *insufficient awareness* (R1, R2), and *concerns about operational burden* (R3, R4). Our analysis adds SBOM-specific nuances: tool vulnerabilities (R4) and platform fragmentation (R3) are barriers that do not apply equally to other security practices. Furthermore, the prevalence of conditional suppression (R1) suggests that many developers view SBOMs as a “nice-to-have” rather than an integral part of their CI/CD pipeline.

Answer to RQ4

RQ4: Why do projects suppress SBOM features in their workflows? Among 363 projects with explicitly documented suppression reasons, we identified four categories of barriers. Conditional and cost-driven suppression (R1, 256 projects) is the most prevalent, followed by incomplete setup (R2, 105), platform incompatibility (R3, 54), and tool vulnerabilities and bugs (R4, 15). These findings suggest that SBOM integration is often deprioritized due to resource constraints and operational burden. These barriers parallel the awareness, applicability, and burden concerns identified for general CI/CD security practices [14].

VIII. TEMPORAL EVOLUTION OF SBOM WORKFLOWS (RQ5)

To understand how SBOM practices evolve over time, we conducted a large-scale longitudinal analysis of workflow commit histories. Using the GitHub Commits API, we retrieved monthly snapshots of each SBOM-related workflow file for 1,851 repositories covering 2,239 workflows. Each snapshot was classified by the LLM into the four usage stages (generation, publication, signing, analysis), with the LLM also providing the specific line numbers used as evidence for each classification. To avoid false positives, we excluded two types of spurious transitions: (1) workflow file deletions (indicating project archival rather than SBOM removal), and (2) single-point classification flickers (a feature detected at only one time point with the feature absent both before and after).

A. ADOPTION AND REMOVAL PATTERNS

Table 10 summarizes the feature-level transition patterns identified across all workflows.

TABLE 10: SBOM feature transition patterns ($n = 2,239$ workflows from 1,851 repositories).

Pattern	Generation	Publication	Signing	Analysis
Feature added	662	349	195	212
Feature removed	156	102	68	56
Net change	+506	+247	+127	+156

While all four features show net positive growth, removal is far from negligible. The overall addition-to-removal ratio is 3.7:1 (1,418 additions vs. 382 removals), indicating that for every feature removed, nearly four are added. However, the ratio varies by feature: generation shows a 4.2:1 ratio, while signing has the lowest at 2.9:1, suggesting that signing is more frequently reconsidered or reverted after initial adoption.

We observed diverse temporal patterns in practice. Some projects, such as `argoproj/argo-events`, progressively added stages and maintained them stably. Others exhibited repeated add/remove cycles (e.g., `owasp-dep-scan/dep-scan`, whose publication feature was added and removed twice between 2022 and 2023) or regressed from multi-stage to fewer stages (e.g., `open-component-model/ocm`, which added analysis in November 2024 to reach three stages but

dropped it by May 2025). A few projects adopted SBOM features and later abandoned them entirely; for instance, `stackabletech/airflow-operator` maintained generation and signing for nearly two years (January 2024 to October 2025) before removing all SBOM features in November 2025.

B. YEAR-BY-YEAR TRENDS

Figure 6 shows the year-by-year feature transitions.

Several trends are noteworthy. First, SBOM generation additions peaked in 2024 (235 additions), coinciding with increased regulatory attention (e.g., the EU Cyber Resilience Act [15]) and maturation of SBOM tooling. Second, removals have accelerated: generation removals rose from 6 in 2022 to 111 in 2025, and the removal-to-addition ratio increased from 12.0% (6/50) in 2022 to 74.5% (111/149) in 2025. This dramatic increase suggests that as more projects experiment with SBOM generation, a growing fraction finds it unsustainable or unnecessary. Third, signing additions showed strong growth from 14 in 2022 to 83 in 2024, reflecting the maturation of Sigstore and GitHub's attestation ecosystem.

These findings demonstrate that SBOM adoption is not a monotonic process. Combined with our RQ4 findings on suppression mechanisms, the data shows that tool vulnerabilities (R4), platform issues (R3), and incomplete setup (R2) continue to drive regression, and that the gap between adoption and sustained usage remains a significant challenge.

C. PHASE PROGRESSION PATTERNS

To characterize how projects evolve across the four SBOM usage stages, we introduce a *phase model* that maps each workflow snapshot to the number of active stages: P0 (no SBOM features), P1 (generation only), P2 (generation + publication), P3 (generation + publication + signing), and P4 (all four stages). A workflow's phase at each time point is defined as the count of active features among generation, publication, signing, and analysis.

Using this model, we identified five typical progression patterns among the 2,239 workflows:

- **Monotonic progression** ($n = 661$, 29.5%): Workflows that only added stages over time, never removing any. These represent the "textbook" adoption trajectory.
- **Stable multi-stage** ($n = 685$, 30.6%): Workflows that maintained a consistent set of SBOM features throughout their observed history, with no phase transitions.
- **Partial regression** ($n = 72$, 3.2%): Workflows that reached a higher phase but later lost one or more stages, ending at a lower phase than their peak.
- **Oscillating** ($n = 137$, 6.1%): Workflows exhibiting repeated phase increases and decreases, reflecting ongoing experimentation or instability.
- **Abandoned** ($n = 220$, 9.8%): Workflows that reached P1 or higher but returned to P0 by the end of the observation period.

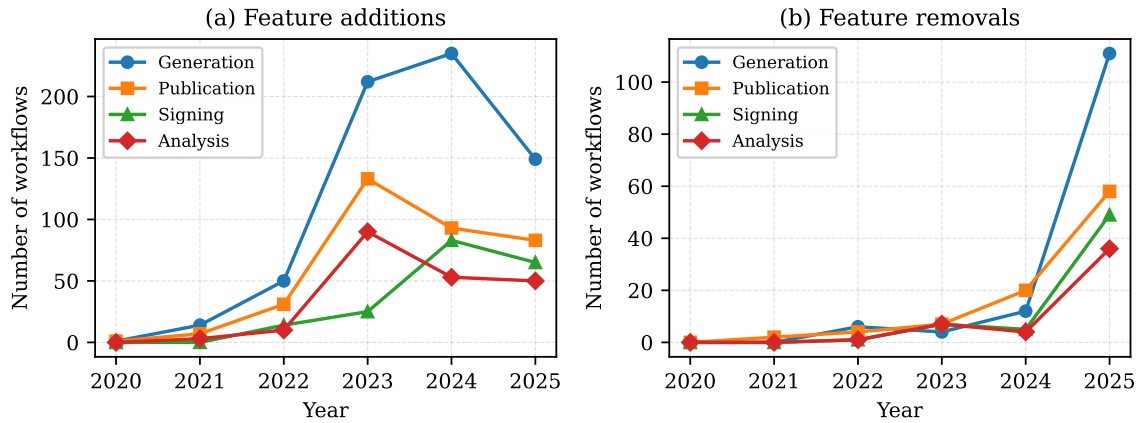


FIGURE 6: Year-by-year SBOM feature transitions (2020–2025). (a) Feature additions per year for each SBOM usage stage. (b) Feature removals per year. 2026 data (January–March only) is omitted for comparability.

The remaining 464 workflows (20.7%) had only a single observed snapshot or exhibited irregular transitions that did not fit any of the five patterns above.

Figure 7 shows representative examples of each pattern. Among the classifiable workflows, monotonic progression (29.5%) and stable multi-stage adoption (30.6%) together account for 60.1% of all workflows, suggesting that the majority of projects, once they establish their SBOM workflow, tend to maintain or expand it. However, the combined share of regression, oscillation, and abandonment (19.2%) indicates that nearly one in five workflows experiences some form of setback in their SBOM practices. The abandonment rate of 9.8% is particularly notable: these workflows invested effort in setting up SBOM steps but ultimately discontinued them entirely.

Answer to RQ5

RQ5: How do SBOM workflows evolve over time? Our longitudinal analysis of 2,239 workflows reveals that SBOM adoption is not monotonic: the addition-to-removal ratio is 3.7:1 overall, but generation removals rose sharply from 6 in 2022 to 111 in 2025. Phase progression analysis shows that 60.1% of workflows follow monotonic or stable trajectories, while 19.2% experience regression, oscillation, or abandonment. Signing has the lowest addition-to-removal ratio (2.9:1), suggesting it is the most frequently reconsidered stage.

IX. DISCUSSION

Our findings have several implications for different stakeholders.

a: For tool developers.

Tool vulnerabilities, bugs, and build errors (R4) directly cause projects to suppress SBOM features, highlighting the need for more robust and well-tested SBOM generation tools. The fact that projects actively regress—removing SBOM steps after encountering issues—means that tool reliability directly impacts long-term adoption. Concretely, CI/CD frameworks

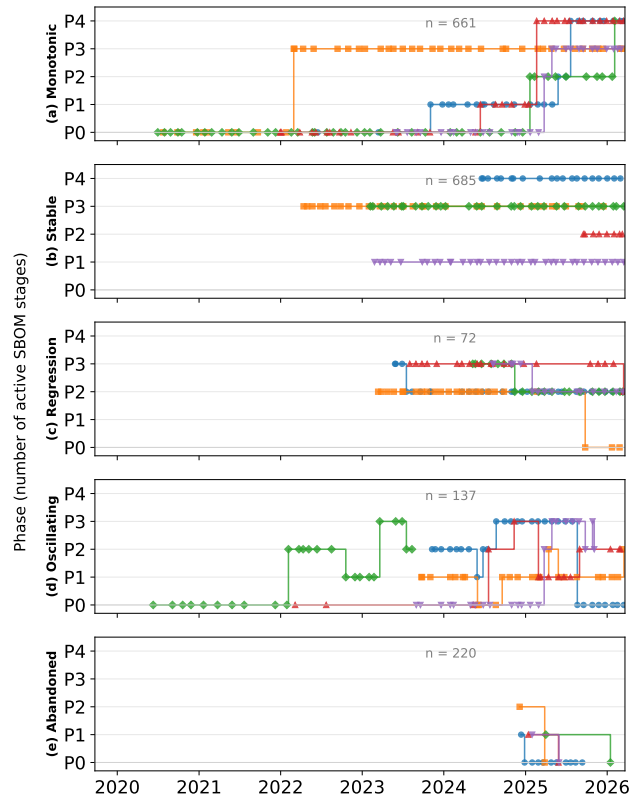


FIGURE 7: SBOM phase progression patterns. Each subplot shows five representative workflows for a given pattern category. The y-axis indicates the phase (P0–P4, i.e., the number of active SBOM stages), and the x-axis spans the observation period.

could incorporate a *path-consistency checker* that statically verifies that SBOM file paths produced in generation steps match the paths consumed in subsequent publication, signing, or analysis steps, thereby catching the most prevalent error category (23.0%) before it causes silent failures. Similarly, an

SBOM stage completeness linter could warn when a workflow generates an SBOM but does not publish or sign it, nudging developers toward multi-stage adoption.

b: For platform providers.

The platform incompatibility barrier (R3), particularly the lack of attestation support in GitHub Enterprise Server, creates a two-tier adoption landscape. Platform providers should ensure feature parity across their offerings to avoid fragmenting the SBOM ecosystem.

c: For policy makers.

The low adoption rate even among critical OSS projects (1.25%) suggests that mandating SBOM generation alone is insufficient. This observation suggests that policies may need to address SBOM *consumption* and *verification* in addition to generation, to ensure that generated SBOMs serve their intended purpose. Our RQ5 results offer indirect evidence that regulatory initiatives may accelerate adoption: SBOM generation additions peaked at 235 in 2024, the year the EU Cyber Resilience Act [15] was formally adopted, following the momentum started by Executive Order 14028 [16] in 2021. However, the concurrent rise in removals (74.5% removal-to-addition ratio in 2025) suggests that policy-driven adoption may not be self-sustaining without adequate tooling and infrastructure support. This concern is amplified by recent policy reversals: in January 2026, the US Office of Management and Budget issued memorandum M-26-05 [17], which revoked the two memoranda (M-22-18 and M-23-16) that had mandated SBOM submission and secure software development attestation for federal software procurement. While agencies may still request SBOMs on a case-by-case basis, the shift from mandatory to discretionary requirements could reduce the incentive for OSS projects to maintain SBOM workflows—potentially accelerating the regression and abandonment patterns we observed in RQ5. Future policies may therefore need to be accompanied by investment in tooling maturity and developer education to sustain adoption beyond regulatory mandates.

d: For researchers.

Our large-scale temporal analysis of 2,239 workflows reveals that SBOM adoption is not monotonic: the generation removal-to-addition ratio has grown from 12.0% in 2022 to 74.5% in 2025, and our phase model shows that 19.2% of workflows experience regression, oscillation, or abandonment. Future work should investigate whether specific events (e.g., Executive Order 14028 [16], NTIA minimum elements publication, the EU Cyber Resilience Act [15]) correlate with adoption spikes or drops, and what factors distinguish projects that achieve stable multi-stage adoption from those that regress.

X. LIMITATIONS

Our study has several limitations. First, our dataset is collected from GitHub code search and may not cover all SBOM-

related workflows, particularly those using custom scripts or private actions that do not match our keyword-based search. Second, we relied on an LLM for workflow classification. Although our manual inspection of 20 samples per stage (80 in total) confirmed all LLM classifications as correct, the sample size is limited relative to the 5,470 workflows classified, and misclassifications on edge cases cannot be ruled out—particularly for boundary cases between image signing and SBOM-specific attestation, or between direct vulnerability scanning and SBOM-based scanning. Furthermore, the LLM's training data likely includes popular GitHub Actions workflows, which may bias its classifications toward well-known patterns and reduce accuracy on uncommon or custom workflows. Because LLM behavior can change across model versions, our results may not be exactly reproducible with a different model snapshot. Additionally, workflows exceeding the model's context window were truncated, potentially causing incomplete classifications for unusually long files. Third, GitHub Code Search, which we used to collect candidate workflows, does not guarantee complete coverage of all public repositories; its indexing scope and result limits may introduce sampling bias, and our dataset should therefore not be assumed to be fully representative of GitHub as a whole. Fourth, our analysis of reasons for suppressing SBOM features is based on comments in workflow files and issue discussions, which may not capture all motivations. Developers who silently remove SBOM steps without leaving comments are not represented in our findings. Finally, we focused exclusively on GitHub Actions as the CI/CD platform. Although GitHub Actions is the dominant CI/CD service for OSS—Golzadeh et al. [11] showed it surpassed Travis CI in under 18 months, Wessel et al. [12] found it covers over 50% of CI-using repositories, and the JetBrains Developer Ecosystem Survey [18] reports approximately 50% developer adoption—SBOM practices on other platforms such as GitLab CI, CircleCI, or Jenkins may differ. Moreover, major SBOM tools (Syft, Trivy, cdxgen, CycloneDX generators) and GitHub's built-in attestation actions (`actions/attest-sbom`) provide first-class GitHub Actions support, which may bias our findings toward tools with strong GitHub integration. Extending this analysis to other CI/CD platforms is left for future work.

XI. RELATED WORK

SBOM Adoption and Stakeholder Perspectives. Xia et al. [19] conducted the first large-scale empirical study of SBOM adoption through 17 interviews and 65 survey responses, finding that 83.1% of participants pointed to the lack of SBOMs for third-party software as a major barrier. Stalnaker et al. [20] surveyed 229 stakeholders across SBOM communities, cyber-physical systems, AI/ML, and legal domains, identifying vulnerability tracking as the primary use case (80.7–100%). Kloeg and van der Linden [21] interviewed 16 industry practitioners and found that system integrators and software vendors are the primary SBOM consumers. Bi et al. [10] analyzed 4,786 GitHub issue discussions

from 510 SBOM-related projects and identified challenges and solutions across the SBOM lifecycle. These studies primarily rely on interviews, surveys, or issue tracker analysis. Our work differs in that we focus on how SBOMs are *implemented* in CI/CD workflows, providing a complementary, code-level perspective on actual adoption.

Nocera et al. [22] mined 186 GitHub repositories for SBOM files and evaluated their NTIA compliance, finding that fewer than half are under version control and that license, hash, and supplier information is frequently missing. Novikov and Riehle [13] found policy-driven SBOM files in only 0.56% of 26,823 repositories. In contrast, our study mines SBOM *workflows* rather than static files; interestingly, our workflow-based adoption rate (1.25%) exceeds their file-based rate (0.56%), suggesting that some SBOM generation processes do not leave persistent files in repositories.

SBOM Quality and Tool Accuracy. Yu et al. [23] conducted a large-scale differential analysis of four major SBOM generation tools and found inconsistencies and missing dependencies in all of them. Halbritter and Merli [24] evaluated SBOM tools against NTIA minimum elements and found that no single tool satisfies all requirements. Soeiro et al. [25] analyzed the diversity and quantity of SBOM files found in public repositories. Our work complements these quality-focused studies by revealing which tools are actually used in practice, providing context for interpreting tool accuracy findings.

Software Supply Chain Security. Ladisa et al. [26] systematically catalogued 107 attack vectors against OSS supply chains and mapped them to 33 safeguards, positioning SBOMs as a transparency-enhancing mitigation. Kuzuno et al. [27] proposed indicators that combine project activity and vulnerability history to gauge security risk. Zahan et al. [28] evaluated the OpenSSF Scorecard across npm and PyPI ecosystems, providing a foundation for ecosystem-wide security measurement that our study builds upon by correlating the Criticality Score with SBOM adoption. The CHAOSS project provides community health metrics, including the Elephant Factor, to evaluate contributor diversity [29].

CI/CD Security. Koishybayev et al. [30] systematically analyzed security risks in CI platforms and showed that 99.8% of GitHub Actions workflows have overly permissive permissions. Riggio and Pautasso [31] analyzed over 200,000 workflow files for nine types of security misconfigurations. Kubo et al. [14] conducted a mixed-methods study of 338,812 repositories and 102 developers, finding that the average implementation rate of five GitHub-recommended security practices is only 17.5%, with the primary barriers being insufficient awareness, misunderstandings about applicability, and concerns about operational burden. These barriers closely parallel our RQ4 findings on SBOM non-adoption. Our study is the first, to our knowledge, to empirically analyze SBOM-specific practices in GitHub Actions at scale.

XII. CONCLUSION

We presented a large-scale empirical study of SBOM usage in GitHub Actions workflows. By mining 3,598 SBOM-generating projects and analyzing the top-10,000 critical OSS projects, we quantified which tools and patterns are used for SBOM generation, publication, signing, and analysis. We showed that SBOM adoption, even among critical projects, remains limited, and that multi-stage SBOM workflows are rare (only 0.5% practice all four stages).

We also identified common mistakes (such as inconsistent file paths) and good practices (multi-format and multi-language SBOMs), as well as practical reasons why projects suppress SBOM features—most commonly conditional or cost-driven suppression and incomplete setup. Our longitudinal analysis of 2,239 workflows further revealed that SBOM adoption is not monotonic: while 60.1% of workflows follow stable or monotonically increasing trajectories, 19.2% experience regression, oscillation, or abandonment. We discussed the implications of these findings for tool developers, platform providers, policy makers, and researchers, aiming to help sustain and improve SBOM adoption in practice.

REFERENCES

- [1] "SOLVING YOUR OPEN SOURCE RISK WITH SOURCECLEAR," VERACODE, 2018.
- [2] S. Ramakrishna, "SolarWinds Security Advisory," 2021.
- [3] Google, "Understanding the Impact of Apache Log4j Vulnerability," in *Google Blog*, 2021.
- [4] NTIA, "The Minimum Elements For a Software Bill of Materials (SBOM)," 2021.
- [5] OpenAI, "OpenAI," <https://openai.com/ja-JP/>, 2025. [Online]. Available: <https://openai.com/ja-JP/>
- [6] B. N. Naveen, "What is the OpenSSF Criticality Score ?" *OpenSSF Blog*, 2023.
- [7] O. Foundation, "CycloneDX," <https://cyclonedx.org/>, 2025. [Online]. Available: <https://cyclonedx.org/>
- [8] T. L. Foundation, "The Software Package Data Exchange," <https://spdx.dev/>, 2025. [Online]. Available: <https://spdx.dev/>
- [9] G. Inc, "GitHub Actions documentation," <https://docs.github.com/en/actions>, 2025. [Online]. Available: <https://docs.github.com/en/actions>
- [10] T. Bi, B. Xia, Z. Xing, Q. Lu, and L. Zhu, "On the Way to SBOMs : Investigating Design Issues and Solutions in Practice," *ACM Transactions on Software Engineering and Methodology*, 2023.
- [11] M. Golzadeh, A. Decan, and T. Mens, "On the rise and fall of CI services in GitHub," in *Proc. IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 662–672.
- [12] M. Wessel, M. Golzadeh, A. Decan, and T. Mens, "GitHub Actions: The impact on the pull request process," *Empirical Software Engineering*, vol. 28, no. 52, 2023.
- [13] A. Novikov and D. Riehle, "Policy-Driven Software Bill of Materials on GitHub: An Empirical Study," in *Proc. International Conference on Product-Focused Software Process Improvement (PROFES)*, 2025.
- [14] Y. Kubo, T. Wakai, and T. Mori, "Action Required: A Mixed-Methods Study of Security Practices in GitHub Actions," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2026.
- [15] European Parliament and Council of the European Union, "Regulation (eu) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (cyber resilience act)," 2024. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>
- [16] The White House, "Executive order 14028: Improving the nation's cybersecurity," 2021. [Online]. Available: <https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/>
- [17] Office of Management and Budget, "Memorandum M-26-05: Software acquisition and use," OMB Memorandum, 2026, <https://www.whitehouse.gov/omb/information-for-agencies/memoranda/>.

- [18] JetBrains, "The state of developer ecosystem 2024," 2024. [Online]. Available: <https://www.jetbrains.com/lp/devecosystem-2024/>
- [19] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu, "An Empirical Study on Software Bill of Materials : Where We Stand and the Road Ahead," in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.
- [20] T. Stalnak, M. D. Penta, D. M. German, and D. Poshyanyk, "BOMs Away ! Inside the Minds of Stakeholders : A Comprehensive Study of Bills of Materials for Software Systems," in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [21] T. Kloeg and D. van der Linden, "Charting the Path to SBOM Adoption," in *Proc. ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2024.
- [22] S. Nocera, E. Di Bella, N. E. D. Ferreyra, S. Bacherini, P. Ferrara, and M.-F. Wendland, "On the Adoption of Software Bill of Materials in Open-Source Software Projects," *Journal of Systems and Software*, 2025.
- [23] S. Yu, W. Song, X. Hu, and H. Yin, "On the Correctness of Metadata-based SBOM Generation: A Differential Analysis Approach," in *Proc. IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024.
- [24] A. Halbritter and D. Merli, "Accuracy Evaluation of SBOM Tools for Web Applications and System-Level Software," in *Proc. International Conference on Availability, Reliability and Security (ARES)*, 2024.
- [25] L. Soeiro, T. Robert, and S. Zaccaro, "Wild SBOMs : a Large-scale Dataset of Software Bills of Materials from Public Code," *arXiv*, 2025.
- [26] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [27] H. Kuzuno, T. Yano, K. Omo, J. V. D. Ham, and Toshihiro Yamauchi, "Security Risk Indicator for Open Source Software to Measure Software," in *Proc. World Conference on Information Security Applications (WISA)*, 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-99-8024-6_12
- [28] N. Zahan, S. Shohan, D. Harris, and L. Williams, "OpenSSF Scorecard: On the Path Toward Ecosystem-Wide Automated Security Metrics," *IEEE Security & Privacy*, 2023.
- [29] S. Goggins, K. Lombard, and M. Germonprez, "Open Source Community Health : Analytical Metrics and Their Corresponding Narratives," in *Proc. IEEE/ACM International Workshop on Software Health (SoHeal)*, 2021.
- [30] I. Koishybayev, A. Nahapetyan *et al.*, "Characterizing the Security of Github CI Workflows," in *Proc. USENIX Security Symposium (USENIX Security)*, 2022.
- [31] L. Riggio and C. Pautasso, "Pipelines Under Pressure: Security Misconfigurations in GitHub Actions Workflows," in *Proc. International Conference on Product-Focused Software Process Improvement (PROFES)*, 2025.

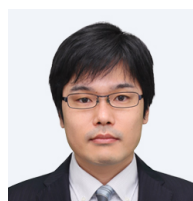


YO KANEMOTO Yo Kanemoto received an M.E. in information science from Nagoya University in 2013 and Ph.D. in informatics from Kyoto University in 2020. Since joining Nippon Telegraph and Telephone Corporation (NTT) in 2013, he has been engaged in research and development on cybersecurity. His research interests include network security and software security.



REIKA NISHIMURA ARAKAWA received the B.I. and M.I in information science from Ochanomizu University, Tokyo, in 2018 and 2020. Since joining the Research and Development Division of NTT, Inc. in 2020, she has been engaged in research and development on supply chain security and cybersecurity. She was awarded the Computer Security Symposium Excellent Paper Award and the IPSJ selected paper award in 2023 and 2025. Her research interests include detecting vulnerable

code and assessing the risk caused by vulnerability.



MITSUAKI AKIYAMA Mitsuaki Akiyama received his M.E. and Ph.D. in engineering from Nara Institute of Science and Technology in 2007 and 2013. Since joining Nippon Telegraph and Telephone Corporation (NTT) in 2007, he has been engaged in research and development on cybersecurity. He is currently a Senior Distinguished Researcher at NTT Social Informatics Laboratories. His research interests include cybersecurity measurement, offensive security, and usable security

and privacy. He is a senior member of IPSJ and a member of IEEE, SIGCHI, and IEICE.

...